Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA

www.dei.estg.ipleiria.pt

Systems Integration

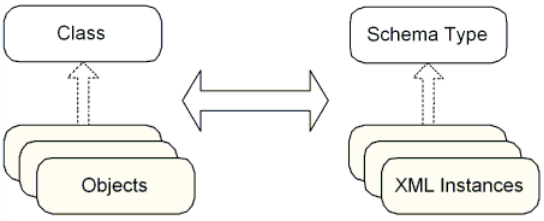
Extensible Markup Language – XML

Goals/Topics:

- Understand the concepts related to XML and its importance in the current information systems
- Identify the main components about the manipulation of information in XML format
- Understand XML Schema definition language (XSD)
- Create an XML Schema (XSD)
- Handle XML though the .NET Core Framework
- Use XPath expressions to query the XML document and retrieve the required information.

Duration: 1,5 classes

©2026-2027: {marisa.maximiano, nuno.costa, humberto.ferreira}@ulo.pt



OO vs XML concepts

– PART 1 –

XML STRUCTURE AND XML SCHEMAS

Based on the XML example shown below:

```
<?xml version="1.0" encoding="utf-8"?>
<bookstore>
  <book category="CHILDREN">
    <title>Charlottes Web</title>
    <author>E.B. White</author>
    <year>1952</year>
    <price>12.60</price>
  </book>
  <book category="WEB">
    <title>Beginning XML</title>
    <author>Joe Fawcett</author>
    <year>2012</year>
    <price>31.30</price>
    <rate>4</rate>
  </book>
  <book category="WEB">
    <title>Programming .NET Web Services</title>
    <author>Alex Ferrara</author>
    <year>2002</year>
    <price>38.36</price>
    <rate>3</rate>
  </book>
</bookstore>
```

Figure 1 – Example of an XML document for a `bookstore`.

1. Identify the existing elements and attributes.

2. Which changes would you make to the XML structure to:

   2.1. Identify the book *publisher*.

   2.2. Allow a book to have several *authors*.

What is a XML Schema (.xsd file)?

The **XML Schema definition language (XSD**) provides a type system for XML processing environments. An XML Schema makes it possible to describe types that you intend to use. An XML document that conforms to an XML Schema type is often referred to as an instance document, very much like the traditional object-oriented (OO) relationship between classes and objects. The schema file also uses the XML notation.

The **<schema>** element is the root of the **XML Schema file (XSD file)**:

```
<?xml version="1.0" encoding="utf-8"?>
<xs:schema ...>
...
...
</xs:schema>
```

Syntax Example for the elements and attributes inside the XSD file:

```
<xs:element name="name" type="xs:string"/>

<xs:attribute name="lang" type="xs:string"/>
```

Complex types example: considering this XML example.

```
<employee>
  <firstname>John</firstname>
  <lastname>Smith</lastname>
</employee>
```

Alternative 1:

```
<xs:element name="employee">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="firstname" type="xs:string"/>
      <xs:element name="lastname" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

A complex element is an XML element that contains other elements and/or attributes.
There are four kinds of complex elements:

- empty elements
- elements that contain only other elements
- elements that contain only text
- elements that contain both other elements and text

Note: Each of these elements may contain attributes as well.

Alternative 2:

```
<xs:element name="employee"
type="personinfo"/>

<xs:complexType name="personinfo">
  <xs:sequence>
    <xs:element name="firstname" type="xs:string"/>
    <xs:element name="lastname" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

In this example, the "employee" element can have a type of attribute that refers to the name of the complex type to use. The advantage is that several elements can refer to the same complex type.

Figure 2 – Example of an XML Schema's content.

3. The objective is to generate an **XML Schema** Definition file (.XSD file) based on an existing XML document (.XML file). Thus, create a new folder inside the solution and complete the following tasks:

3.1. Add a new XML file named `bookstore.xml` and enter (or copy and past) the XML content shown in Figure 1.

3.2. Then, use an online XML-to-XSD converter tool, such as https://www.liquid-technologies.com/online-xml-to-xsd-converter , to generate the corresponding XSD Schema Definition file from the `bookstore.xml`. Verify that the generated schema is equivalent to the one obtained in **Alternative 1.** Save the resulting file as `bookstore.xsd`.

3.3. Analyse the generated `bookstore.xsd` schema and identify the definition of all **elements** and **attributes**. Indicate which XSD tags are use (e.g. `xs:element`, `xs:attribute`) and try to understand their purpose within the schema.

3.4. Look at the auto-generated file. What is a complex type?

3.5. What is the difference between `<sequence>`, `<choice>` and `<all>`?

3.6. Where are the elements' details changed? Namely:

- The number of *minimum occurrences* and *maximum occurrences*.
- The element type.

3.7. Modify the generated XML Schema so that its structure matches **Alternative 2.** Define **named complex types** (`xs:complexType`) instead of anonymous complex types, enabling their reuse throughout the schema.

3.8. Extend the schema to support the following new requirements: each book element may contain a **bookformat** child element and also an **isbn** attribute.

3.9. Modify the XML Schema to enforce the following constraints:

3.9.1. The **category** attribute must accept only the following values: WEB, CHILDREN, SCIENCE, ROMANCE, etc.

3.9.2. The **rate** element must accept only integer values in the range 1 to 5.

3.9.3. The **year** element must contain exactly 4 digits.

3.9.4. The **bookformat** element must accept only the following values: paperback, electronic book text, hardback, ebook, loose-leaf.

4. Auto-generate a new XML file based on the changed XML Schema(XSD file). The aim is to test the changes implemented in the new XML Schema File.

   4.1. For example, use an online Schema validation and generation tool, such as https://extendsclass.com/xml-schema-validator.html, to generate a sample XML document from your XSD schema.

   4.2. Check the status message displayed in the green/red validation bar below the editor. If the XML document is valid, the tool will indicate that the XML conforms to the specified XSD Schema. Next, modify the year element by replacing it with a larger value that does not satisfy the schema restriction (for example, a value containing more than four digits).

– PART 2 –

MANIPULATING XML USING DOM AND XPATH

Create a new solution named **SolutionXML** and add a new **Avalonia .NET APP** project named **ProjectXML.** As an alternative, you may download the base project provided on the course web page and add it to the solution. All exercises in this assignment must be implemented in C#.

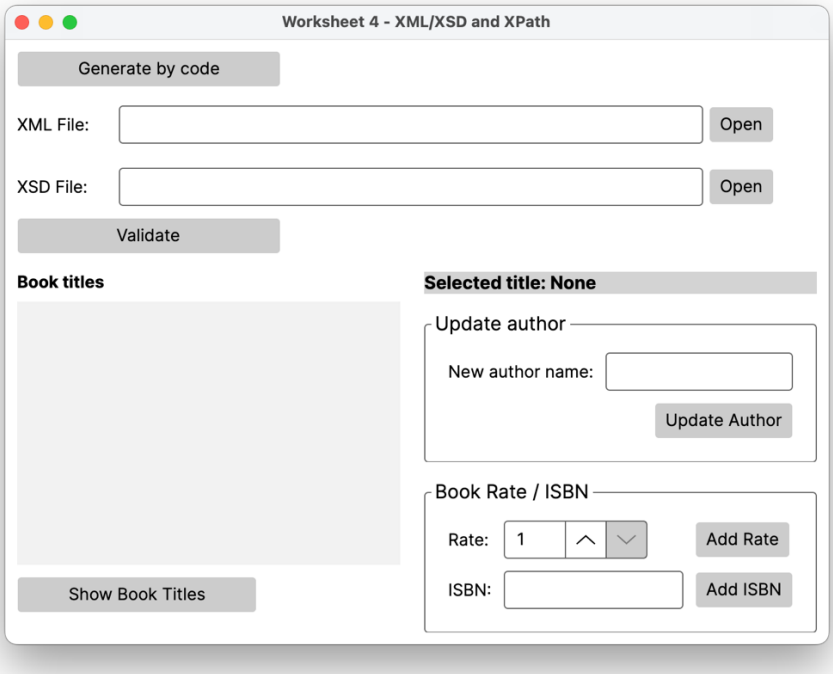


Figure 3 – Expected user interface for the project. A starter version of the project is provided on the course web page.

5. Develop a function that creates an XML document that conforms to the structure presented in Figure 1. The document must be generated programmatically using the DOM API available in the `System.Xml` namespace. All elements, attributes, and values must be created in code. A simplified example illustrating the XML generation is provided in Figure 4.

5.1. Add a **Generated by code** button to the `MainWindow` and implement the corresponding event handler to execute the XML generation function.

DOM (*Document Object Model*): Example showing how to create the XML file in memory using DOM:

```
        XmlDocument doc = new XmlDocument();

        // Create the XML Declaration, and append it to XML document
        XmlDeclaration dec = doc.CreateXmlDeclaration("1.0", null, null);
        doc.AppendChild(dec);

        // Create the root element
        XmlElement root = doc.CreateElement("bookstore");
        doc.AppendChild(root);
        // Create Books
        // Note that to set the text inside the element,
        // you use .InnerText
        // You use SetAttribute to set attribute
        XmlElement book = doc.CreateElement("book");
        book.SetAttribute("category", "CHILDREN");
        XmlElement title = doc.CreateElement("title");
        title.InnerText = "Chalottes Web";
        XmlElement author = doc.CreateElement("author");
        author.InnerText = "E.B. White";
              ...
    book.AppendChild(title);
    book.AppendChild(author);
    root.AppendChild(book);
             ...

    string xmlOutput = doc.OuterXml;
    Console.WriteLine(xmlOutput);

    // OR save the XML in the disc to test in the next exercise
     doc.Save(@"example.xml");
```

…the output is as follows:

```
<?xml version="1.0"?>
<bookstore>
  <book category="CHILDREN">
    <title>Charlottes Web</title>
    <author>E.B. White</author>
  </book>
</bookstore>
```

Figure 4 – Example code using .NET DOM functionalities to create an XML document.

5.2. Examine the `HandlerXML.cs` file provided with the project. It contains example code that can be used as a starting point and as reference when completing the following exercises.

6. Implement a function that checks whether an XML document is **well formed** and **valid** according to a given XML Schema (XSD). The validation must be performed using the `Xml.XmlDocument` and `ValidationEventHandle`. Any validation errors or warnings during the process must be displayed.

Tip: Refer to the documentation of the `XmlDocument` class and examine its available methods and properties. Some of them can significantly simplify the implementation of schema-based validation.

7. Implement a function that reads the XML document and retrieves the **titles** of all books stored in it. Display the retrieved titles in a **ListBox** control on the application's user interface.

8. Implement a function that update the **author** of a book identified by its title. After the update, save the modified XML document to preserve the changes. To test the function, allow the user to: select a book title from the **ListBox** populated in the previous exercise, enter the new author's name in the **TextBox** control, and execute the update operation through a button on the user interface.

9. Implement a function that adds a **rate** element to a specific book in the XML document. The target book must be identified by its **title**. The value of the **rate** element should be provided by the user through the application's user interface (e.g. **NumericUpDown**). After adding the element, save the modified XML document to preserve the changes.

10. Implement a function that adds an **isbn** attribute to a specific book. The target book must be identified by its title. The value of the isbn attribute should be provided by the user. After adding the attribute, save the modified XML document.

[ADDITIONAL EXERCISES]

11. Add a button to the `MainWindow` that displays the number of books in a specific **category**. Implement a function that receives the category name as an argument and returns the number of books belonging to that category. Display the result in the application user

12. Figure 5 illustrates an XPath editor used to test XPath expressions. Create a new **Avalonia .NET App** project named **XPathEditor** that replicates the interface shown in the figure. The application must provide: a text box for entering an XPath expression, a text box for specifying the path to an XML file, an "Execute" button to run the XPath query, a results area where the matching nodes or values are displayed.

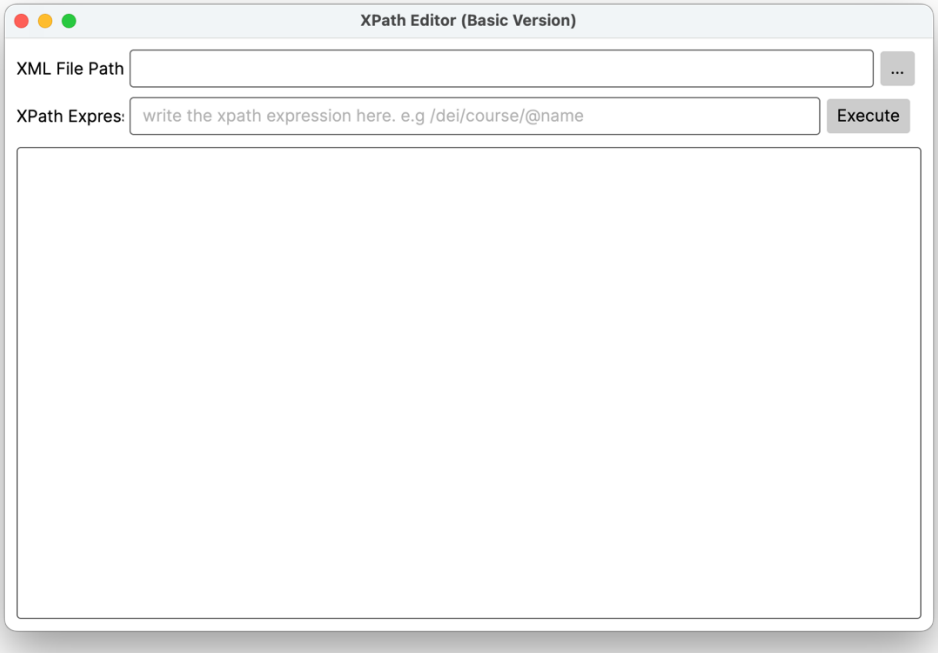


Figure 5 – Expected user interface for the XPath editor application.

13. The Department of Informatics Engineering stores information about the courses offered each semester in an XML format. Download the two sample files, named **`dei_sample.xml`** and **`dei_xample.xsd`** from the course web page.

Using the information contained in the XML file, write the **XPath** expressions required to retrieve the following data:

13.1. Select the first course in the XML file.

13.2. Retrieve the name of the second course.

13.3. Retrieve the name of the last course.

13.4. Retrieve the name of the second-to-last course.

13.5. Retrieve the name of the first teacher of the first course.

13.6. Retrieve the names of the teachers who teach in the IS course.

13.7. Retrieve the names of all courses.

13.8. Retrieve the student number of the students whose name contains “Manuel”.

13.9. Retrieve the student number of the student whose name starts with “M”.

13.10. Retrieve the course regime from the courses whose named start with “I” and have 6 ECTS credits.

13.11. Retrieve the names of all the courses associated with the teacher named “Francisco”.

13.12. Retrieve the ECTS credits of all courses.